

Docker & PostgreSQL — A Beginner's Cheat Sheet

Run, manage, back up, and troubleshoot PostgreSQL in Docker — explained step by step. If you've never opened a terminal before, start at the top and follow each section in order.

1. What is Docker, and why use it for Postgres?

Docker is a tool that runs software inside lightweight, isolated boxes called **containers**. A container bundles the application + everything it needs (libraries, config, data paths) so it runs the same way on your laptop, your colleague's machine, and the production server.

For a database admin, this means you can spin up a fresh PostgreSQL instance in seconds, experiment freely, throw it away, and never pollute your host machine with leftover files or services.

[i] Three words you'll keep hearing

Image = a frozen template (e.g. `postgres:17`). **Container** = a running copy of an image. **Volume** = a folder Docker manages so your data survives even if the container is deleted.

2. 10 terms you'll see on every page

Term	What it means (in plain English)
image	Read-only template used to create containers. Versioned by tag (e.g. <code>:17</code>).
container	A running instance of an image. You can start, stop, and delete it.
volume	Persistent storage managed by Docker. Use this for your database files.
bind mount	Maps a folder on your host machine into the container.
port mapping	<code>-p 5432:5432</code> means: connect to port 5432 on your laptop → reach Postgres inside the container.
environment variable	Settings passed in with <code>-e KEY=value</code> (e.g. <code>POSTGRES_PASSWORD</code>).
network	A virtual LAN that lets containers talk to each other by name.
Docker Compose	A tool that runs <i>multiple</i> containers from a single YAML file. Same idea, less typing.
healthcheck	A small command Docker runs to decide if your container is healthy or sick.
registry	Where images live (Docker Hub by default). <code>docker pull</code> downloads from here.

3. Your very first PostgreSQL container

Open a terminal and type the command below. Every line is explained in the next box — don't worry about understanding every flag yet.

```
docker run -d \
  --name pg17 \
  -e POSTGRES_PASSWORD=secret \
  -e POSTGRES_USER=app \
  -e POSTGRES_DB=appdb \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

Line-by-line explanation

<code>docker run -d</code>	Start a new container in the background (-d = detached).
<code>--name pg17</code>	Give the container a memorable name. Without this, Docker invents one like <i>elated_kepler</i> .
<code>-e POSTGRES_PASSWORD=secret</code>	Set the superuser password. Required — Postgres refuses to start without it.
<code>-e POSTGRES_USER=app</code>	Create a non-root user named 'app'. Optional; defaults to 'postgres'.
<code>-e POSTGRES_DB=appdb</code>	Create an initial database. Optional; defaults to the username.
<code>-p 5432:5432</code>	Expose port 5432 on your host so tools like psql/pgAdmin can connect.
<code>-v pgdata:/var/lib/postgresql/data</code>	Save data to a Docker-managed volume named 'pgdata'. This is what keeps your data safe.
<code>postgres:17</code>	The image to use — Postgres major version 17. Other tags: 16, 15, 17-alpine.

[!] Never use a real password like 'secret' in production

It's fine for learning. For anything that touches real data, use Docker secrets, a password manager (Vault, AWS Secrets Manager), or — at minimum — a long random string passed in via a file.

4. Check it's actually running

```
# 1. Is the container alive?
docker ps                                # shows running containers

# 2. Watch the logs (Ctrl+C to stop watching, container keeps running)
docker logs -f pg17

# 3. Connect with psql from inside the container
docker exec -it pg17 psql -U app -d appdb
# you should land in a prompt like:
# appdb=#
```

[*] If 'docker ps' is empty

The container probably crashed during startup. Run `docker ps -a` (note the -a) to see stopped containers, then `docker logs pg17` to read the error. Most failures are: missing password, port already in use, or a permission problem on the volume.

5. Container lifecycle — the everyday commands

Command	What it does
<code>docker ps</code>	List running containers.
<code>docker ps -a</code>	List all containers, including stopped ones.
<code>docker logs -f --tail 100 pg17</code>	Show last 100 log lines and follow new ones.
<code>docker exec -it pg17 psql -U app appdb</code>	Open a psql shell inside the container.
<code>docker exec -it pg17 bash</code>	Open a Linux shell inside the container.
<code>docker stop pg17</code>	Gracefully stop the container (Postgres shuts down cleanly).
<code>docker start pg17</code>	Start a stopped container again. Data is preserved.
<code>docker restart pg17</code>	Restart in place — same as stop + start.
<code>docker rm pg17</code>	Delete the stopped container. Volume data still survives.
<code>docker stats pg17</code>	Live CPU, memory, and disk usage.

6. Volumes — keeping your data safe

A container is **throwaway**. If you don't tell Docker where to store the data, it disappears when the container is removed. The fix is a **named volume**: a folder Docker manages that outlives any single container.

[i] How -v actually works

`-v pgdata:/var/lib/postgresql/data` means: take the volume named **pgdata** (create it if it doesn't exist) and mount it at the path **/var/lib/postgresql/data** inside the container. That path is exactly where Postgres stores its data files.

```
# List all volumes on your machine
docker volume ls

# See where Docker keeps the volume on disk
docker volume inspect pgdata

# Delete a volume (WARNING: this destroys the data!)
docker volume rm pgdata
```

[!] The most common beginner mistake

Skipping the `-v` flag. The container starts, you create tables, you load data — then you run `docker rm` and everything is gone. Always use a named volume from day one.

7. Networking — letting containers talk to each other

By default, every container is on a shared default network and is reachable on the host. When you start running multiple containers (e.g. an app + a database), put them on a **user-defined network** so they can find each other by name.

```
# 1. Create a network
docker network create pgnet

# 2. Start Postgres on that network
docker run -d --name pg17 --network pgnet \
  -e POSTGRES_PASSWORD=secret -v pgdata:/var/lib/postgresql/data postgres:17

# 3. Run a one-off psql client on the same network
# (no host port needed — it talks to 'pg17' directly)
docker run --rm -it --network pgnet postgres:17 \
  psql -h pg17 -U postgres
```

8. What is Docker Compose?

Typing long docker run commands gets old fast. **Docker Compose** lets you describe one or more containers in a single YAML file — then start them all with one command.

[i] compose.yaml = your stack as code

Save the file as `compose.yaml` (or `docker-compose.yml`) in any folder. Run `docker compose up -d` from that folder. Commit it to Git so anyone on your team can reproduce the exact same setup.

9. Your first compose.yaml — fully annotated

```
# compose.yaml — a single Postgres service for local development
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_USER: app
      POSTGRES_DB: appdb
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./initdb:/docker-entrypoint-initdb.d:ro
    ports:
      - "5432:5432"
    shm_size: "1gb"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

volumes:
  pgdata:
```

short name: 'db' (used by other containers)
the Postgres image
auto-restart unless YOU stopped it
same as -e flags from earlier
named volume for data
any .sql or .sh file in ./initdb runs ONCE on first startup
map host:container
bigger shared memory = better perf
Docker checks the DB is alive
declare the named volume

10. The Compose commands you'll actually use

Command	What it does
<code>docker compose up -d</code>	Start everything in the background.
<code>docker compose ps</code>	Status of each service (running, healthy, exit code).
<code>docker compose logs -f db</code>	Tail logs from one service.
<code>docker compose exec db psql -U app appdb</code>	Open psql inside the running 'db' service.
<code>docker compose pull</code>	Download newer image versions without starting.
<code>docker compose up -d</code>	Re-run after a pull to recreate with the new image.
<code>docker compose stop</code>	Stop containers but keep them. Restart with <code>start</code> .
<code>docker compose down</code>	Stop and remove containers. Volumes are KEPT — your data is safe.
<code>docker compose down -v</code>	DESTRUCTIVE : also delete named volumes. You will lose data.
<code>docker compose config</code>	Validate the YAML and show the merged config.

11. Backups — the most important page in this guide

[!] A backup that hasn't been restored doesn't exist

Take backups daily, but **practice restoring** them at least once a month. If you've never run a restore, you don't have a backup — you have a hope.

Logical backup (recommended for small/medium databases)

pg_dump reads the data through SQL and writes a portable file. Easy to inspect, easy to restore selectively, slower for very large databases.

```
# Back up a single database to a custom-format file (best for selective restore)
docker exec -t pg17 pg_dump -U app -Fc -d appdb -f /tmp/appdb.dump

# Copy the dump out of the container to your host machine
docker cp pg17:/tmp/appdb.dump ./appdb-$(date +%F).dump

# Or stream it directly to a file on your host (no copy step)
docker exec -t pg17 pg_dump -U app -Fc appdb > appdb.dump

# Back up ALL databases + roles + tablespaces (cluster-wide)
docker exec -t pg17 pg_dumpall -U postgres > cluster.sql
```

Restoring a logical backup

```
# Create the target database first (if it doesn't exist)
docker exec -it pg17 createdb -U app appdb_new

# Restore from the dump file (--clean drops objects before re-creating them)
docker exec -i pg17 pg_restore -U app -d appdb_new --clean --if-exists < appdb.dump
```

[*] Custom format vs. plain SQL

Use -Fc (custom). It's compressed, lets you list contents with pg_restore -l, and supports parallel restore (-j). Plain SQL only wins when you need to *read* the dump.

12. A simple read replica with Compose

Once you're comfortable with one Postgres, the next step is a primary + read replica. Below is a minimal stack — copy it into a new compose.yaml and run docker compose up -d.

```
services:
  primary:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
      POSTGRES_USER: app
    command: >
      postgres -c wal_level=replica -c max_wal_senders=10 -c hot_standby=on
    volumes:
      - primary-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro # creates 'replicator' role
    networks: [pgnet]

  replica:
    image: postgres:17
    depends_on: [primary]
    environment:
      PGUSER: replicator
      PGPASSWORD: replpass
    entrypoint: >
      bash -c '
        until pg_isready -h primary -U replicator; do sleep 2; done;
        if [ -z "$(ls -A /var/lib/postgresql/data)" ]; then
          pg_basebackup -h primary -D /var/lib/postgresql/data -U replicator -Fp -Xs -P -R;
        fi;
        exec docker-entrypoint.sh postgres'
    volumes: [replica-data:/var/lib/postgresql/data]
    networks: [pgnet]

networks: { pgnet: {} }
volumes: { primary-data: {}, replica-data: {} }
```

Create `./init/01-replicator.sql` with: `CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'replpass';` — this runs once on first start and creates the replication user.

13. When things go wrong — a beginner's playbook

What you see	Likely cause	Try this
Error: bind: address already in use	Another program (often a local Postgres or a leftover container) is using port 5432.	Change the host port: <code>-p 5433:5432</code> , or stop the other service.
FATAL: password authentication failed	Wrong password, or you re-created the container with a new password but kept the old volume.	Volumes are remembered. Either use the original password, or delete the volume to start fresh.
role "app" does not exist	You changed POSTGRES_USER but the volume already had data initialised with the old user.	Init scripts only run on a <i>new</i> data dir. Drop the volume to re-init.
container exits with code 137	The container was OOM-killed (out of memory).	Raise the limit: <code>--memory=2g</code> or in compose <code>deploy.resources</code> .
could not translate host name "db"	Containers aren't on the same user-defined network.	Put both services in the same networks: block, or use Compose (it auto-creates one).
disk full / could not extend file	Volume is on a filesystem that's run out of space.	Free space on the host (Docker Desktop has its own VM disk you may need to grow).
database system was interrupted	Container was killed mid-write. Postgres is recovering on next start.	Wait for docker logs to show 'database system is ready'.

14. Five investigation commands for any Postgres-in-Docker problem

Command	What it does
<code>docker logs --tail 200 pg17</code>	Read the most recent log lines first. 90% of issues are visible here.
<code>docker exec pg17 pg_isready -U app</code>	Is Postgres accepting connections? Exit code 0 = yes.
<code>docker exec -it pg17 psql -c 'SELECT * FROM pg_stat_activity;'</code>	Who's connected, what queries are running, are any blocked.
<code>docker exec pg17 du -sh /var/lib/postgresql/data</code>	How much disk is the database using?
<code>docker inspect pg17 --format '{{.State.OOMKilled}}'</code>	Was the container killed for using too much memory? <i>true</i> = yes.

15. What to learn next

- **pg_basebackup** for physical backups, and Point-in-Time Recovery (PITR).
- **PgBouncer** in front of Postgres for connection pooling.
- **Prometheus + Grafana + postgres_exporter** for monitoring inside Compose.
- **Docker secrets** instead of plain-text passwords in YAML.
- Graduating from Compose to **Kubernetes** with the CNPG operator for automated failover and PITR.

Two rules to take with you. (1) The container is throwaway — the volume is sacred. (2) Pin your image version, set a healthcheck, take backups, and rehearse restores. Everything else builds on those four habits.

Continue learning with TDA — visit www.thedbadmin.com for hands-on labs and mentorship. Questions? admin@thedbadmin.com • +91 6232090055.